ヒューマノイドの重量物押し制御における強化学習方策の検討

（omnicampus アカウント名:141978）

PAI 最終課題 説明資料

概要

腕付きの二足ヒューマノイド（Unitree G1 の派生モデル）が、10〜40 kg の箱を押しながら歩くタスクを MuJoCo 上で扱った。歩行計画、DCM 安定化、全身トルク制御、手先の力制御からなる共通の制御基盤を構築し、その上位指令の決め方が異なる 3 方式を比べた。学習を用いない Analytic、8 種類の上位指令を PPO で学習する RL-M1、押付け力と前傾の 2 指令を学習する RL-M2 である。各方式 12 試行（4 質量 × 3 試行）の評価で、成功数は Analytic 7、RL-M1 8、RL-M2 10 であった。ただし RL-M1 と RL-M2 は行動数だけでなく報酬の設定も異なるため、本レポートは方策構成の検討と位置づける。

1　目的と課題設定

重い箱をヒューマノイドで押すには、箱を進める力と、押し返されても倒れない姿勢を両立させる必要がある。箱からの反力は胴体を後ろへ押し戻すので、強く押すほど身体の前後の釣り合いが崩れやすい。

本課題の目的は、既存の制御技術と強化学習を組み合わせ、ヒューマノイドが重量物を押せるようにすることである。また、共通の制御基盤に対して、強化学習に任せる上位指令の種類を変えたときに、箱押しの成功数と動作がどう変化するかを調べる。

■推論（強化学習）を使用する箇所　PPO[1] で学習したニューラルネットワークの方策が、実行時に 10 Hz で上位指令（押付け力、前傾など）を決める。方策は箱の質量や摩擦係数を観測せず、手首の力覚センサの値と箱の動きから指令を決める。

■工夫した点（設計上の狙い）

- 強化学習の担当範囲を上位指令に限定するため、歩行と力制御を共通の制御基盤として構築した。学習する範囲を小さくすることで、短期間で学習を完了でき、かつ学習なしの制御基盤は比較基準となると考えた。
- 強化学習の設定として、多自由度のタイプ RL-M1、前傾姿勢を意図させた RL-M2 を用意した。RL-M1 は補正できる自由度を増やして成功するかを調べることを狙った。一方で、RL-M2 は人が重量物を押すときの姿勢から、押付け力に合わせて身体を前傾させれば成功しやすくなると考え、前傾の指令を学習させることを狙った。

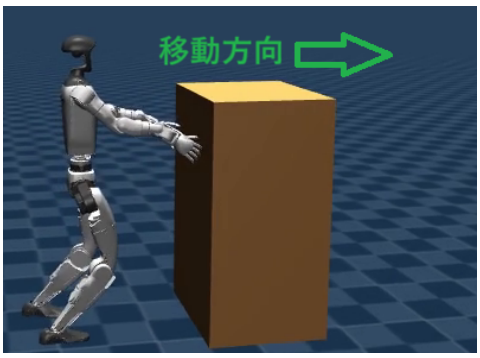


図 1　タスクの全体像。ロボットは前方の箱（0.5 × 0.6 × 1.0 m）を両手で押しながら、矢印の向きに 0.1 m/s で前進する。

2　タスクと実験条件

シミュレータには MuJoCo 3.12[2] を用いた。ロボットには、MuJoCo Menagerie[3] の Unitree G1 を元に、腰の 3 関節を固定し、手の接触と手首 6 軸力覚センサを加えた派生モデル（脚 12 関節、腕 14 関節、33.3 kg）を用いた。制御周期は 2 ms（500 Hz）、学習方策の判断周期は 100 ms（10 Hz）である。箱はロボットの前方 0.71 m に置き、時刻 2 s から 0.1 m/s の前進指令を与えて、10 s で 1 試行を終える。

成功は、10 s の終了時に箱が初期位置から前方へ 0.5 m 以上移動しており、途中でロボットの転倒も箱の転倒も起きていないことと定めた。ロボットの転倒は胴体のロールまたはピッチが 40 deg を超えたとき、箱の転倒は箱のロールまたはピッチが 20 deg を超えたときとし、その時点で試行を打ち切る。したがって、箱を 0.5 m 以上動かしていても、途中で転倒した試行は失敗である。

学習時と評価時の条件を表 1 に示す。主評価では、各質量で 3 試行を行う。箱と床の摩擦係数は試行ごとに 0.15〜0.35 から選ぶが、乱数の種を固定しているため、3 試行の摩擦係数はそれぞれ 0.277、0.252、0.202 となる。この値は 3 方式と 4 質量のすべてで共通であり、3 方式は同じ条件の組で比べている。学習時も摩擦係数は同じ 0.15〜0.35 から箱ごとに乱択しており、方策は摩擦係数を観測しない。したがって主評価の 3 値は、学習した範囲の中にあるが学習中に同じ値が現れたわけではない摩擦係数であり、その摩擦係数でも押せるかを見る標本として用いた。これは学習分布の内側での汎化を少数の値で確かめたものであり、学習範囲の外の摩擦係数は評価していない。これとは別に、指令と応答の時系列を示す代表例（5.2 節）は 30 kg、摩擦係数 0.2 の各方式 1 試行であり、主評価の 36 試行には含まれない。

表 1　学習時と評価時の条件

条件	Analytic	RL-M1	RL-M2
学習	なし	PPO、20 並列環境、30 万判断	
学習時の箱の質量	–	10〜40 kg を一様乱択	
学習時の摩擦係数	–	0.15〜0.35 を一様乱択	
学習時の箱の初期位置	–	前後 0〜0.04 m のずれを一様乱択	
主評価の質量	10、20、30、40 kg、各 3 試行（1 方式 12 試行、計 36 試行）		
主評価の摩擦係数	0.277、0.252、0.202（各質量の 3 試行に対応、3 方式で共通）		
主評価の箱の初期位置	ずれなし（設定値の位置に固定）		
代表例（時系列）	スナップショットのみ（図 7）	30 kg、摩擦係数 0.2、各 1 試行	

3　制御系の構成

3.1　共通の制御基盤

3 方式は、図 2 の共通の制御基盤を通してロボットを動かす。基盤は 4 層からなり、上位方策の層だけが方式ごとに異なる。

歩行計画の層は、速度指令から足配置と支持相を決め、DCM（付録 A）の参照軌道を作る。DCM フィードバックの層は、推定した DCM と参照の差から足裏の CoP 指令と重心に要求するレンチを作る [5]。全身トルク制御の層は、要求レンチから手で受け持つ分（手首力覚センサの実測反力）を差し引き、残りを二次計画で両足の接触レンチに配分してから、受動性に基づく全身制御で関節トルクに写像する [6, 7]。手先は、押す向きだけを力のフィードフォワードと実測反力の積分補正で制御し、横方向と高さは位置で、手の姿勢は保持するように制御する。押す向きを力制御にすると、箱が急に滑ったときに手が箱を追って腕が伸び切るので、骨盤から手先までの距離に上限を設けて押付け力を絞る。

上位指令は、10 Hz で値が飛ぶことを避けるため、変化率に上限（押付け力 100 N/s、前傾 0.2 m/s）を掛けてから下位層に渡す。本文の図で用いる量は次のとおりである。

- **押付け力指令** f_{push}：両手で箱を押す力の指令値の合計 [N]。手首力覚センサで測った両手の反力の合計を実測値として比べる。
- **前傾指令** Δ_{lean}：歩行計画の CMP 参照を足に対して前へずらす量 [m]。胴体の角度の指令ではない。

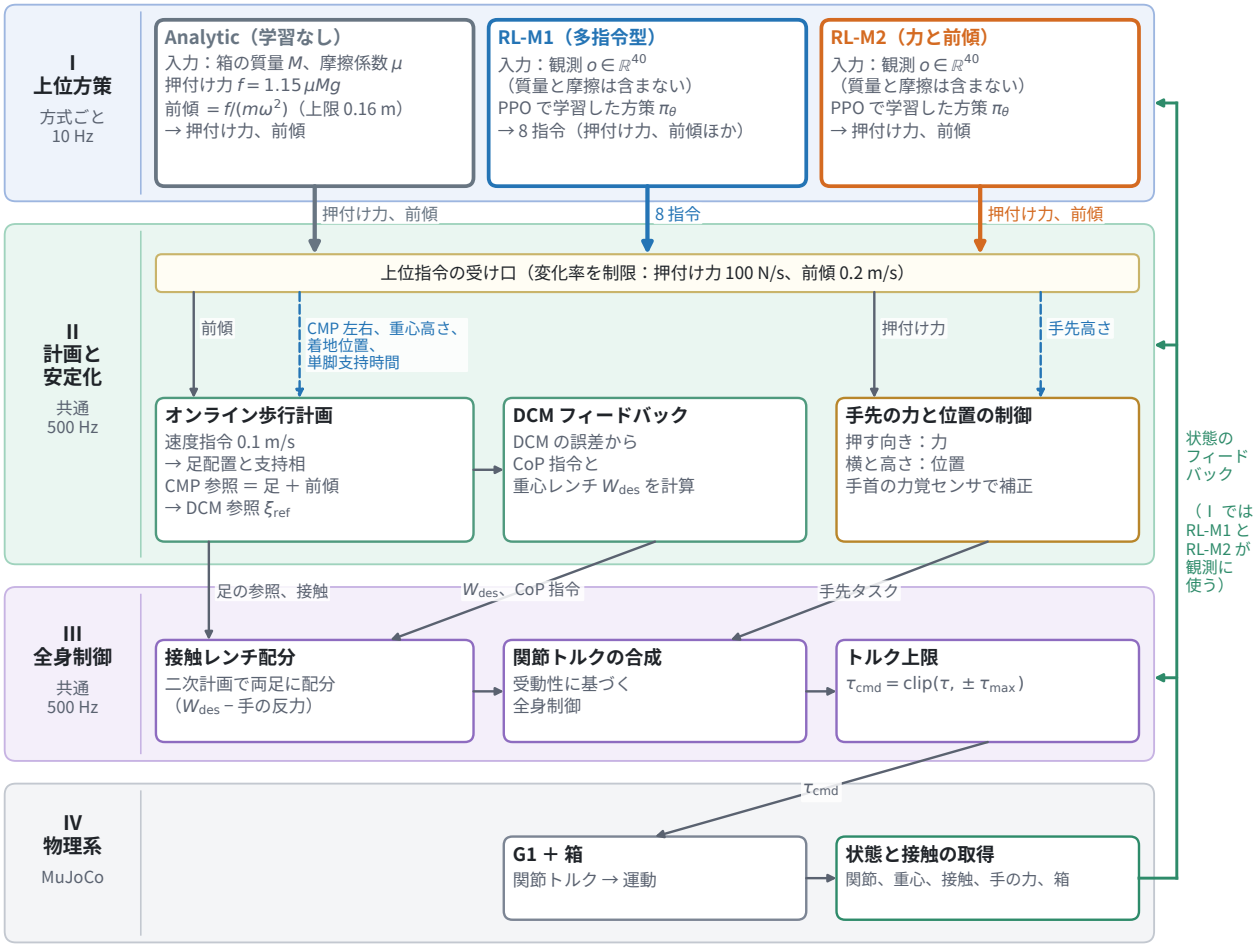


図 2　制御系の構成。I の上位方策だけが方式ごと（Analytic、RL-M1、RL-M2）に異なり、II〜IV は 3 方式で共通である。上位方策の指令は II の受け口で変化率を制限してから、歩行計画と手先の制御に入る。実線は 3 方式に共通の指令、青の破線は RL-M1 だけが出す指令。

前傾指令を大きくすると、重心が足より前に出た状態で歩くことになる。ただし重心を前へ出す方法は胴体を傾けるだけでなく、股関節や足首でも作れるので、前傾指令と胴体の角度は一対一に対応しない（図 6）。

DCM と CMP の定義と、前傾指令の大きさの目安は付録 A にまとめる。

3.2　比較する 3 方式

3 方式の違いを表 2 にまとめる。説明の順は、基準となる Analytic、多指令型の RL-M1、力と前傾に絞った RL-M2 とする。

表 2　方式別の利用情報と上位指令

項目	Analytic	RL-M1（多指令型）	RL-M2（力と前傾）
上位指令の決め方	解析則と手設計のスケジュール	PPO で学習した方策	PPO で学習した方策
箱の質量と摩擦係数	押付け力の計算に用いる	方策に入力しない	方策に入力しない
方策の観測	–	40 次元	40 次元
学習する指令	なし	押付け力、手先高さ、CMP 前後、CMP 左右、重心高さ、着地位置の前後と左右、単脚支持時間（8 出力）	押付け力、前傾（2 出力）
前傾の決め方	$f_{\mathrm{push}}/(m\omega^2)$	CMP 前後の出力	前傾の出力
判断周期	10 Hz（前傾は毎制御周期に計算）	10 Hz	10 Hz

■Analytic（学習なし） 押付け力の目標を、箱の質量と摩擦係数から $f_{\mathrm{push}} = 1.15\,\mu M g$（$M$ は箱の質量）とし、時刻 2 s から 1 s かけて立ち上げる。前傾は付録 A の釣り合いから $\Delta_{\mathrm{lean}} = f_{\mathrm{push}}/(m\omega^2)$ とし、上限を 0.16 m とする。押す前（0.5 s 以降）に 0.05 m だけ先に前傾し、押付け力は「現在の前傾で支えられる力 +12 N」を超えないように制限する。箱の質量と摩擦を知っている点で、学習方策より多くの情報を使う比較基準である。

■RL-M1（多指令型） 押付け力（0〜120 N）と手先高さ（±0.15 m）に加え、歩行計画への補正として CMP 前後（±0.20 m）、CMP 左右（±0.10 m）、重心高さ（±0.15 m）、着地位置の前後と左右（各 ±0.10 m）、単脚支持時間（±0.10 s）の計 8 指令を出す。CMP 前後の出力は RL-M2 の前傾と同じ入力口に入るので、RL-M1 にも前傾の自由度はある。PPO で学習した方策が、モデルベースの制御基盤に 8 種類の上位指令を出す構成である。

■RL-M2（力と前傾） 押付け力（0〜120 N）と前傾（±0.20 m）の 2 指令だけを出す。ほかの量は固定値とし、計画層の補正は学習しない。

両方式の方策の観測は 40 次元で共通である。内訳は、歩行の上位方策の観測 29 次元（DCM の誤差と予測、支持相、足の相対位置、重心の位置と速度、速度指令、胴体の傾きなど）と、箱押しの 11 次元（箱の面までの相対位置と箱の速度、左右の手の実測反力、手先高さ、重心高さ、現在の CMP オフセット、関節トルクの飽和率、押付け力指令）である。

4　学習方法と評価関数

4.1　学習の設定

学習には Stable-Baselines3[4] の PPO を用いた。方策と価値関数はそれぞれ隠れ層 64 ユニット 2 層の tanh 多層パーセプトロンで、出力を $[-1, 1]$ に制限してから各指令の範囲に写像する。学習率 3×10^{-4}、ロールアウト長 256、ミニバッチ 256、エポック数 10、割引率 $\gamma = 0.99$、GAE の $\lambda = 0.95$、クリップ幅 0.2 とし、観測と報酬は移動平均で正規化した。両方式とも 20 並列環境で 30 万判断を学習した。

4.2　評価関数（報酬関数）

1 回の判断（100 ms、制御 $N = 50$ 周期）あたりの報酬 R_k は、制御周期 j ごとの項を $\Delta t = 2\,\mathrm{ms}$ で積算した値に、行動の変化と終了時の項を加えたものである。

$$
\begin{aligned}
R_k = \sum_{j=1}^{N} \Big[& w_{\mathrm{prog}}\, v_{\mathrm{box},j} + w_{\mathrm{con}} \mathbb{1}[f_{\mathrm{meas},j} \geq 0.8\, f_{\mathrm{cmd},j}] - w_{\mathrm{sat}}\, \rho_{\mathrm{sat},j} - w_{\mathrm{slip}} \sum_{i \in \mathcal{C}_j} \| \boldsymbol{v}^{xy}_{\mathrm{foot},i} \| \\
& - w_{\mathrm{dcm}} \, \| \hat{\boldsymbol{\xi}}_j - \boldsymbol{\xi}_{\mathrm{ref},j} \|^2 - w_{\mathrm{post}} \, (\phi_j^2 + \theta_j^2) \Big] \Delta t - w_{\mathrm{act}} \, \| \boldsymbol{a}_k - \boldsymbol{a}_{k-1} \|^2 - W_{\mathrm{fall}} \mathbb{1}_{\mathrm{fall}} - W_{\mathrm{tip}} \mathbb{1}_{\mathrm{tip}} + W_{\mathrm{succ}} \mathbb{1}_{\mathrm{succ}}
\end{aligned}
\tag{1}
$$

各項の意味と係数を表 3 に示す。押している判定 $\mathbb{1}[\cdot]$ は、押付け力指令が 1 N を超え、実測反力がその 8 割以上のときに 1 とする。転倒と箱の転倒の項は発生した判断で 1 回だけ与えて試行を終え、成功の項は 10 s の終了時に成功条件を満たしたときに与える。前傾の量や重心の高さそのものは報酬に含めていない。

両方式の差は、行動の数（8 と 2）と姿勢の項の係数 w_{post}（0 と 1）の二つである。したがって、成功数の差を「指令を 2 つに絞ったこと」だけに帰着させることはできない。

4.3　性能評価の指標

報酬とは別に、方式の比較には次の指標を用いる。

- **成功率**：成功した試行数 / 試行数。各方式 12 試行。
- **箱の最終変位**：試行終了時（転倒による打ち切りを含む）の箱の前後方向の移動量 [m]。
- **平均押付け力指令、平均前傾指令**：代表例では保存した時刻 2 s 以降のサンプルの平均。主評価では時刻 3 s 以降の平均。
- **DCM 誤差 RMS**：$\sqrt{\frac{1}{T}\sum_t \| \hat{\boldsymbol{\xi}}_t - \boldsymbol{\xi}_{\mathrm{ref},t} \|^2}$ [mm]。代表例の時刻 2 s 以降で集計する補助指標。

表 3　報酬の項と係数

係数	項の意味	RL-M1	RL-M2
w_{prog}	箱の前進速度 v_{box} [m/s]	10	10
w_{con}	押している（実測反力が指令の 8 割以上）	0.5	0.5
w_{sat}	関節トルクの飽和率 ρ_{sat}（上限の 95% 以上の関節の割合）	2	2
w_{slip}	接地中の足 $\mathcal{C}$ の水平速度 [m/s]	5	5
w_{dcm}	DCM 誤差の二乗 [m^2]	10	10
w_{post}	胴体のロール ϕ とピッチ θ の二乗 [rad^2]	**0**	**1**
w_{act}	行動（$[-1,1]$ に正規化）の変化の二乗	0.05	0.05
W_{fall}	ロボットの転倒	50	50
W_{tip}	箱の転倒	20	20
W_{succ}	成功	20	20

時刻 2 s は歩行開始の設定時刻であり、押し始めの時刻ではない。実際の接触と箱の移動はそれ以前にも起きている（図 5）。

5　結果

5.1　質量別の成功数

主評価 36 試行の結果を表 4 と図 3 に示す。10 kg と 20 kg では 3 方式とも全試行で成功し、差は 30 kg と 40 kg で生じた。RL-M2 は 30 kg で 3/3、40 kg で 1/3 成功し、合計 10/12 で最も多かった。

表 4　質量別の成功数（成功数/試行数）と失敗の内訳。各質量の 3 試行は、摩擦係数 0.277、0.252、0.202 に対応する。

方式	10 kg	20 kg	30 kg	40 kg	合計	転倒	箱の転倒	平均最終変位 [m]
Analytic	3/3	3/3	1/3	0/3	7/12	5	0	0.556
RL-M1	3/3	3/3	2/3	0/3	8/12	4	0	0.685
RL-M2	3/3	3/3	3/3	1/3	**10/12**	2	0	0.836

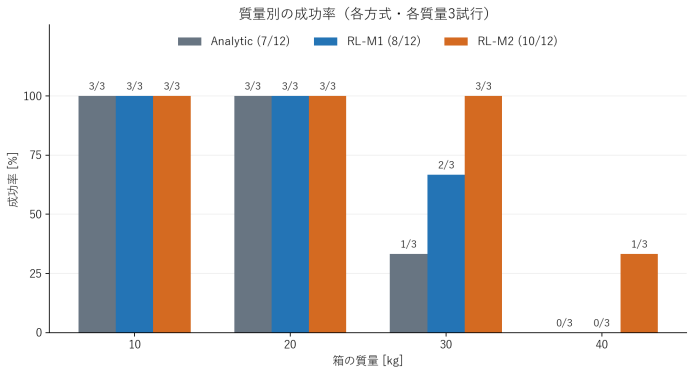


図 3　質量別の成功率。各方式、各質量 3 試行（1 方式 12 試行）。棒の上の数字は成功数/試行数。

失敗はすべてロボットの転倒であり、箱の転倒は起きなかった。Analytic の転倒は 3.3〜5.8 s と押し始めて間もない時刻に起き、5 試行とも前傾指令の平均が上限 0.16 m に達していた。RL-M1 と RL-M2 の転倒は 5.1〜9.0 s に起きた。

5.2　指令と応答の代表例

30 kg、摩擦係数 0.2 の代表例で、RL-M1 と RL-M2 の指令と応答を比べる（図 4、図 5）。両方式とも成功した試行である。

図 4 は押付け力の指令と、手首力覚センサで測った両手の反力の合計である。指令した力がそのまま箱に伝わるわけではなく、実測値は歩行の周期に合わせて大きく変動し、指令を下回る時間が長い。歩行に伴う手先の前後の揺れや、箱が滑り出したときに反力が抜けることが影響している可能性がある。

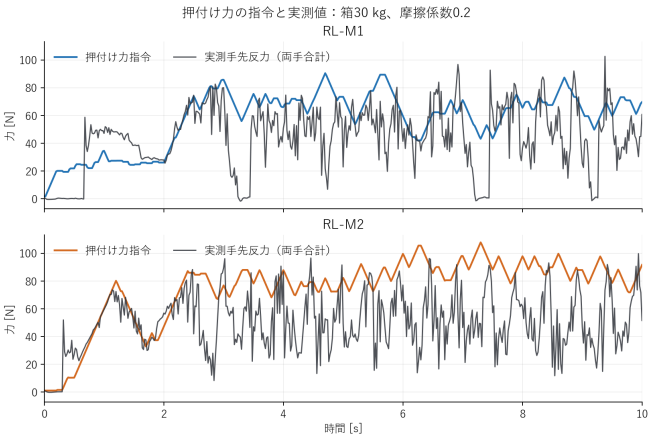


図 4　押付け力の指令と実測反力（両手合計）。

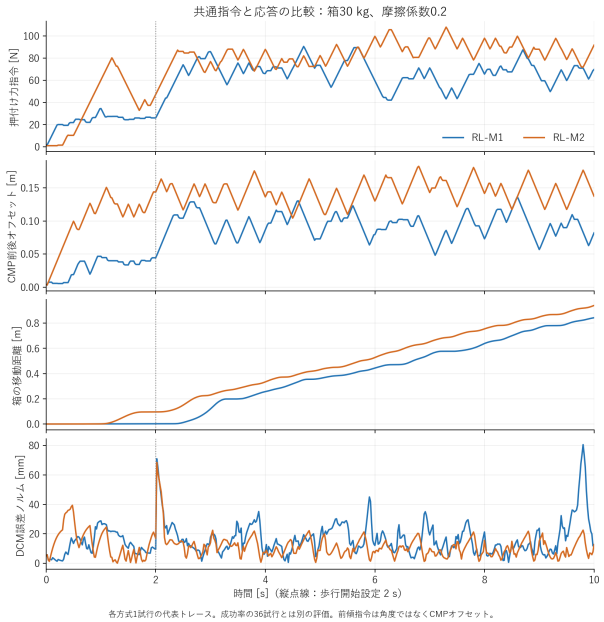


図 5　指令と応答の時系列。縦の点線は歩行開始の設定時刻。前傾指令は胴体の角度ではなく CMP 参照の前後オフセット。

図 4、図 5 とも、箱 30 kg、摩擦係数 0.2 の代表例（各方式 1 試行、主評価の 36 試行とは別の評価）。

図 5 は、上から押付け力指令、前傾指令（CMP 前後オフセット）、箱の移動距離、DCM 誤差である。RL-M2 は試行開始の直後から前傾と押付け力をともに大きくし、歩行開始前に箱を約 0.1 m 動かした。時刻 2 s 以降の平均は、押付け力指令が RL-M1 の 66.2 N に対して RL-M2 は 84.2 N、前傾指令が 0.094 m に対して 0.145 m であった。最終的な箱の移動距離は 0.843 m に対して 0.940 m、DCM 誤差 RMS は 20.9 mm に対して 13.4 mm であった。

図 6 は、同じ代表例の胴体のピッチとロール、重心と両足中点の前後距離である。胴体の角度は骨盤の姿勢から求めた値で、腰の関節を固定しているので胴体と一体に動く。RL-M2 は前傾指令が大きいにもかかわらず、時刻 2 s 以降の胴体ピッチは平均 −0.1 deg、RMS 1.1 deg で、ほぼ直立を保っていた。一方で、重心と両足中点の前後距離の平均は 0.142 m で、前傾指令の平均 0.145 m とほぼ等しかった。前傾指令は胴体を傾けることではなく、重心を足より前に置くことで実現されていたと読める。RL-M1 も重心の前後距離（平均 0.090 m）は前傾指令（平均 0.094 m）に近かったが、胴体ピッチは最小 −11.5 deg（後ろに傾く向き）、ロールは −8.1〜5.8 deg と大きく揺れた。

図 7 は、同じ 30 kg、摩擦係数 0.2 の条件で 3 方式の動作を同じ時刻に並べたものである。RL-M1 と RL-M2 は図 4〜図 6 と同じ試行で、どちらも 10 s まで押し続けた。Analytic はこの条件の記録が無かったため、同じ設定で新たに記録した。Analytic は 8 s までに箱を 0.66 m 動かしたが、8.6 s で転倒した。この試行は主評価の 36 試行とは別の 1 試行であり、方式の優劣は表 4 で比べる。

6　考察と限界

6.1　学習した指令の違いと結果

30 kg の代表例では、RL-M2 の押付け力指令と前傾指令の平均がともに RL-M1 より大きく、箱の最終変位も大きかった。主評価の記録でも同じ傾向があり、30 kg の 3 試行で RL-M2 の前傾指令の平均は 0.143〜0.165 m、RL-M1 は 0.095〜0.110 m であった。DCM 誤差 RMS は RL-M2 のほうが小さかったが、これは各方式の参照に対する追従誤差であり、RL-M2 のほうが転倒しにくいことを直接示すものではない。

一方、押付け力と前傾は同時に変化しており、どちらが成功数の差に効いたかは今回の比較からは分離できない。加えて、RL-M1 と RL-M2 では行動数と報酬の姿勢項が異なる。このため、観測された指令と応答の違いは報告できるが、成功数の差の原因は特定できず、RL-M2 の構成が一般に優れているとも結論できない。

Analytic は、箱の質量と摩擦を知っていても 30 kg 以上で成功が減った。失敗した試行では前傾指令が上限 0.16 m

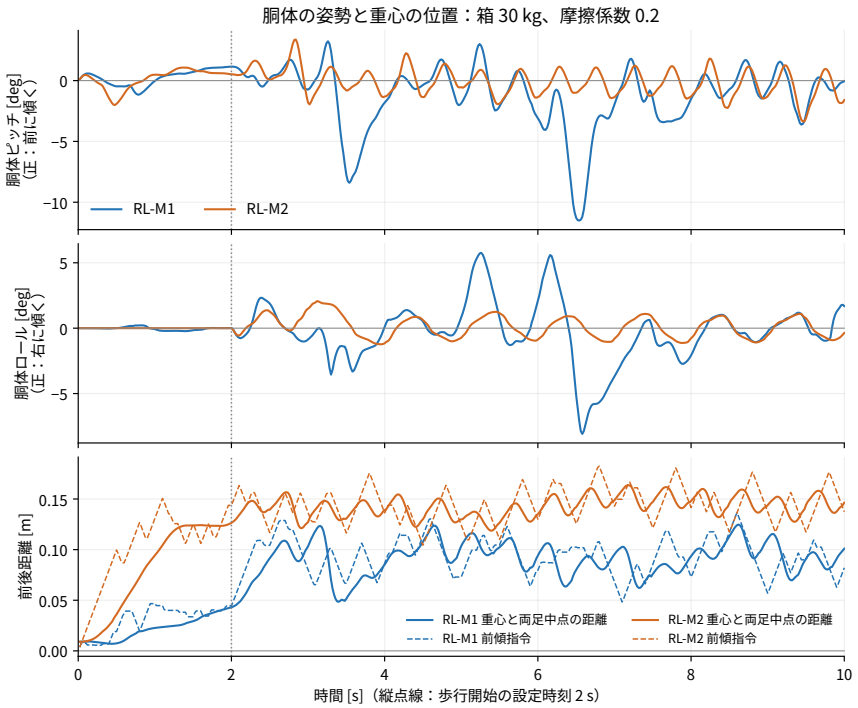


図 6　胴体の姿勢と重心の位置。箱 30 kg、摩擦係数 0.2 の代表例（図 4、図 5 と同じ試行）。下段の実線は重心と両足中点の前後距離、破線は前傾指令。

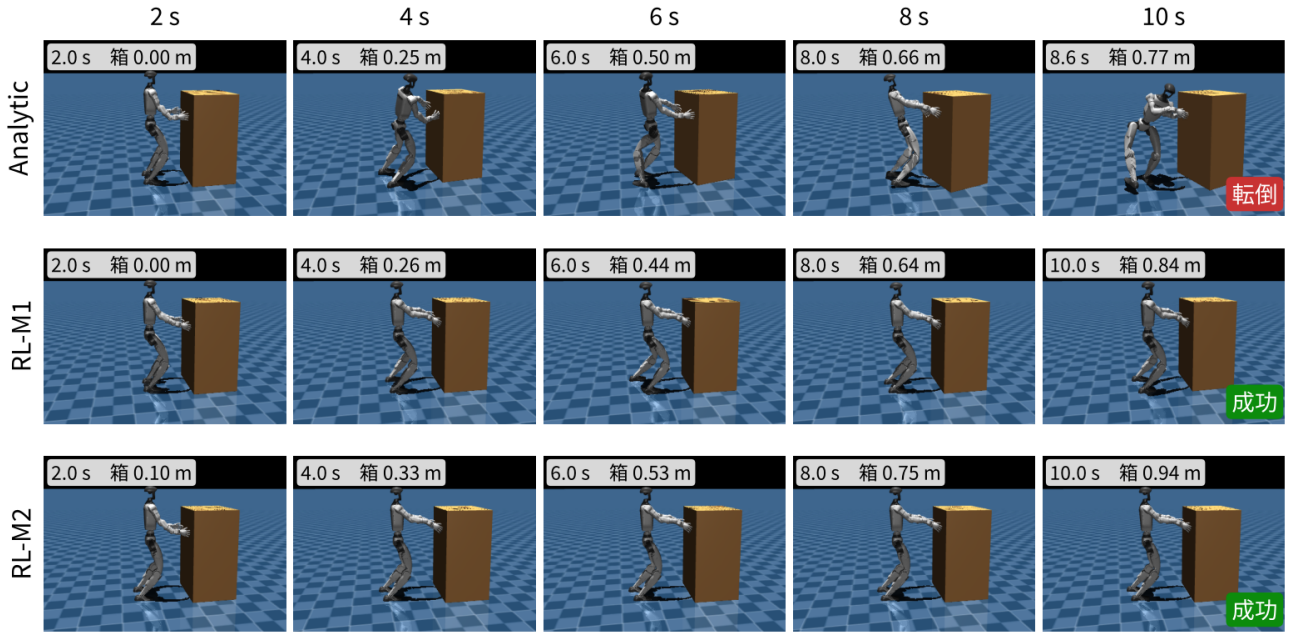


図 7　3 方式の動作の比較（箱 30 kg、摩擦係数 0.2、各方式 1 試行）。列は時刻、各コマの左上は描いた時刻と箱の移動距離。Analytic は 8.6 s で転倒したため、右端の列は転倒時の姿を示す。

に達しており、釣り合いに必要な前傾（たとえば 30 kg、摩擦係数 0.277 で約 0.18 m）を取れないまま押していた。前傾の上限と、それに伴う押付け力の制限が転倒に関わった可能性があるが、今回の評価だけでは確定できない。

6.2　本検討の限界と今後の課題

本課題では、限られた期間の中で、ヒューマノイドによる重量物押しに対する強化学習方策を検討した。そのため、評価した方策、学習条件、箱の質量、試行回数はいずれも限定的である。今回の評価では RL-M2 が最も多く成功したが、各質量 3 試行の結果であり、この結果だけから RL-M2 が常に優れているとは結論できない。

今後は、報酬と学習条件をそろえた比較、試行回数の追加、摩擦係数と初期位置を広く変えた評価が必要である。特に摩擦係数は、学習範囲（0.15〜0.35）の内側の 3 値でしか評価しておらず、範囲の外での振る舞いは分かっていない。押付け力と前傾を個別に変えるアブレーション実験を行えば、それぞれの指令が成功率に与える影響を切り分けられる。また、本モデルはシミュレーション用の派生モデルであり、40 kg、摩擦係数 0.2 で必要な押付け力（静止摩擦で約 78 N）は押し切るための目安にすぎず、実機の能力を保証するものではない。実機での検証と、未知の条件に対する汎化性能の評価は今後の課題である。

7　結論

共通の制御基盤の上で、上位指令の決め方が異なる Analytic、RL-M1、RL-M2 を比べた。今回の条件では、押付け力と前傾の 2 指令を学習する RL-M2 が 12 試行中 10 試行で成功し、最も多かった。指令の与え方によって押付け力と前傾の使い方に違いが現れたが、各要素の寄与の切り分けは今後の課題である。

参考文献

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.

[2] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ IROS*, pp. 5026–5033, 2012.

[3] K. Zakka, Y. Tassa, and MuJoCo Menagerie Contributors, “MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo,” `https://github.com/google-deepmind/mujoco_menagerie`, 2022.

[4] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[5] J. Englsberger, C. Ott, and A. Albu-Schäffer, “Three-Dimensional Bipedal Walking Control Based on Divergent Component of Motion,” *IEEE Trans. Robotics*, vol. 31, no. 2, pp. 355–368, 2015.

[6] G. Mesesan, J. Englsberger, G. Garofalo, C. Ott, and A. Albu-Schäffer, “Dynamic Walking on Compliant and Uneven Terrain using DCM and Passivity-based Whole-body Control,” in *Proc. IEEE-RAS Humanoids*, pp. 25–32, 2019.

[7] B. Henze, A. Dietrich, and C. Ott, “An Approach to Combine Balancing with Hierarchical Whole-Body Control for Legged Humanoid Robots,” *IEEE Robotics and Automation Letters*, vol. 1, no. 2, pp. 700–707, 2016.

[8] M. B. Popovic, A. Goswami, and H. Herr, “Ground Reference Points in Legged Locomotion: Definitions, Biological Trajectories and Control Implications,” *The International Journal of Robotics Research*, vol. 24, no. 12, pp. 1013–1032, 2005.

[9] S. Kajita, F. Kanehiro, K. Kaneko, K. Yokoi, and H. Hirukawa, “The 3D Linear Inverted Pendulum Mode: A simple modeling for a biped walking pattern generation,” in *Proc. IEEE/RSJ IROS*, pp. 239–246, 2001.

[10] J. Pratt, J. Carff, S. Drakunov, and A. Goswami, “Capture Point: A Step toward Humanoid Push Recovery,” in *Proc. IEEE-RAS Humanoids*, pp. 200–207, 2006.

付録 A　DCM と CMP の補足

重心高さ z_c を一定とみなす線形倒立振子では、手が箱から後ろ向きの反力 f を受けるとき、重心の前後位置 c の運動は

$$\ddot{c} = \omega^2 (c - r_{\mathrm{cmp}}) - \frac{f}{m}, \qquad \omega = \sqrt{g/z_c}$$

で表せる [9]（m はロボットの質量）。ここで r_{cmp} は CMP（Centroidal Moment Pivot）であり、重心を通り床反力と平行な直線が床と交わる点である [8]。重心まわりのモーメントが無いとき、CMP は足裏の CoP と一致する。

本基盤の歩行計画は、手の反力の項をまとめた $r_{\mathrm{ref}} = r_{\mathrm{cmp}} + f/(m\omega^2)$ を **CMP 参照**として扱う。すると運動は $\ddot{c} = \omega^2(c - r_{\mathrm{ref}})$ と、手の反力が無いときと同じ形になる。DCM（Divergent Component of Motion）は $\xi = c + \dot{c}/\omega$ で定義され、$\dot{\xi} = \omega(\xi - r_{\mathrm{ref}})$ に従う [5]。DCM は重心の運動のうち放置すると発散する成分だけを取り出した量であり、capture point[10] と同じ点を指す。本基盤の DCM フィードバックは、$p_{\mathrm{cmd}} = \hat{\xi} + k(\hat{\xi} - \xi_{\mathrm{ref}}) - \dot{\xi}_{\mathrm{ref}}/\omega$ により CoP 指令を決める。

静止して押し続けるとき、重心と DCM は CMP 参照の真上にあり、足裏の CoP は CMP 参照より $f/(m\omega^2)$ だけ後ろになる（図 8）。前傾指令が無いと CMP 参照は足の中心にあるので、CoP は足の中心から後ろへずれる（図 8(a)）。前傾指令は CMP 参照を足の中心から Δ_{lean} だけ前へずらす。$\Delta_{\mathrm{lean}} = f/(m\omega^2)$ とすれば、CoP は足の中心に留まったまま、重心が足より前に出る（図 8(b)）。本モデルでは $m = 33.34\,\mathrm{kg}$、$\omega = 3.90\,\mathrm{rad/s}$ であり、$m\omega^2 \approx 507\,\mathrm{N/m}$ となる。たとえば 80 N で押し続けるのに釣り合う前傾は約 0.16 m で、足裏の長さ（約 0.17 m）に近い。前傾が無ければ CoP は約 0.16 m 後ろにずれてかかとの外に出るので、その姿勢では押し続けられない。

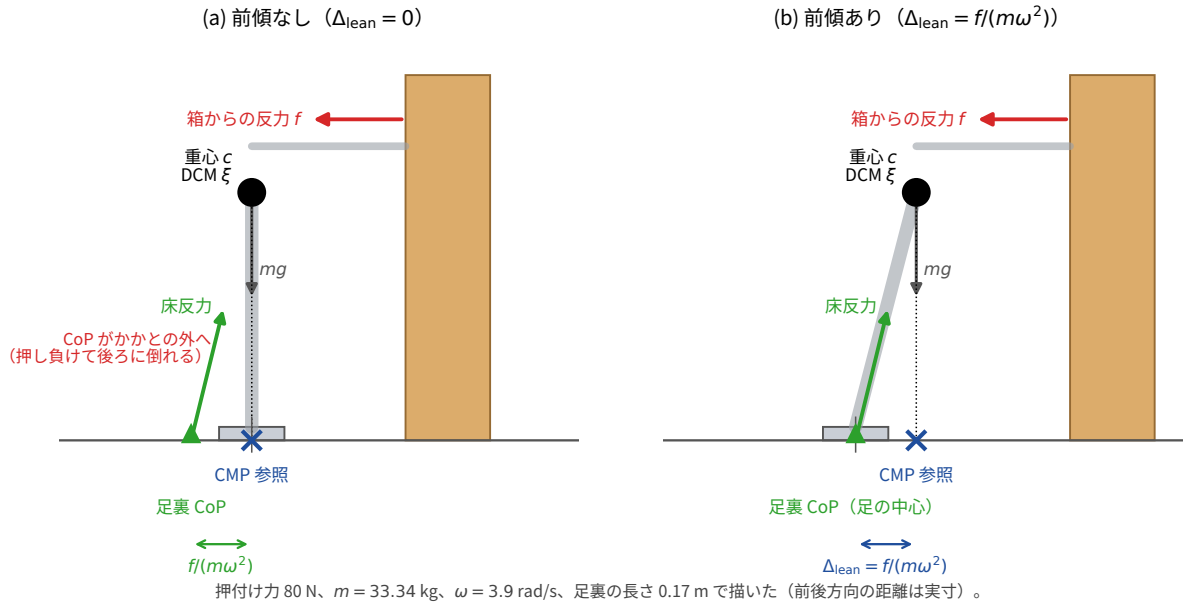


図 8　前傾指令の役割を示す側面図（押付け力 80 N で静止して押し続ける場合）。× は CMP 参照、▲ は足裏 CoP、黒丸は重心と DCM（静止時は一致）。(a) 前傾なしでは CoP がかかとの外に出る。(b) 前傾指令で CMP 参照を前へずらすと、CoP は足の中心に留まり重心が前に出る。

5 章で示した **DCM 誤差** $\|\hat{\boldsymbol{\xi}} - \boldsymbol{\xi}_{\mathrm{ref}}\|$ は、各方式が自分で作った参照に対する追従誤差であり、転倒しにくさを直接測る量ではない。

付録 B　実行コードと再現手順

コード、設定、学習済みモデル、再実行手順は次のリポジトリに掲載する。

- GitHub：`https://github.com/xonak/physical-ai-applied1-2026`
- 動画：`https://youtu.be/ShN3hf7S7lk`

動画の代表例は、本資料の代表例（30 kg、摩擦係数 0.2）と同じ条件で再収録した別試行である。そのため、動画の箱の移動量は本資料の図と表の値と一致しない。

主評価は、提出用リポジトリの `armtasks/boxpush` で次のコマンドを実行すると再現できる（環境の構築手順はリポジトリの README に記載）。学習済みの方策は `models/rl_m1`（RL-M1）と `models/rl_m2`（RL-M2）に同梱している。本資料の結果は手の接触を従来のメッシュ形状としたモデルで得たため、`--robot robot_g1_box` を指定する。

```
python -m boxpush.cli.eval_policy box_p2_walk_push --robot robot_g1_box \
  --policies analytic,../../models/rl_m1,../../models/rl_m2 \
  --labels Analytic,RL-M1,RL-M2 --masses 10,20,30,40 --seeds 3 \
  --friction 0.15,0.35 --x-offset 0,0 --out ../../outputs/eval_main.jsonl
```

■生成 AI の利用　プログラムの作成、本資料と動画の作成には、Claude（Anthropic）と ChatGPT（OpenAI）を使用した。評価の数値は、すべて保存した評価ログと時系列ファイルから集計した。

■利用した既存技術　制御則は DCM 歩行制御 [5] と受動性に基づく全身制御 [6, 7] の構成に基づく。シミュレータに MuJoCo[2]、ロボットモデルに MuJoCo Menagerie の Unitree G1[3]、学習に Stable-Baselines3[4] を用いた。

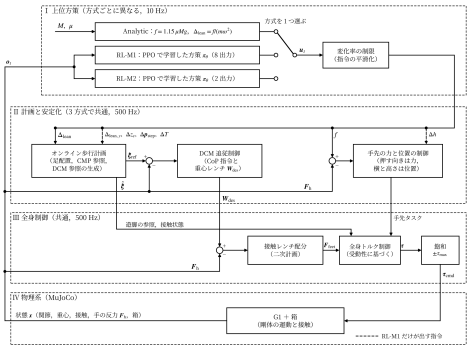


図 9　タスクの全体像を示す制御ブロック図。